

Company: Microsoft

Interview Type: TECHNICAL

Q1: Hierarchical Resource Locking

Difficulty: Hard

Tags: N-ary Tree, Concurrency, API Design

Problem Statement: Design an N-ary tree data structure to manage resource locking. Each node represents a resource. Implement three functions: `lock(node, user_id)`, `unlock(node, user_id)`, and `upgrade(node, user_id)`.

A node can be locked if and only if:

1. The node is not already locked.
2. No ancestor of the node is locked.
3. No descendant of the node is locked.

`upgrade` allows a user to lock a node if they already hold the locks on all currently locked descendants, and no ancestors are locked. Upon success, all descendant locks are released.

Expected Approach: Use a basic N-ary tree where each node stores its lock status and the `user_id`. For `lock`, traverse up to the root to check ancestors and perform a DFS/BFS to check descendants.

Optimized Approach: Augment each node with a `locked_descendant_count` and a `parent` pointer.

1. `lock`: Check `locked_descendant_count == 0` and traverse `parent` pointers to ensure no ancestor is locked ($O(H)$ where H is height).
2. `unlock`: Reset status and decrement `locked_descendant_count` for all ancestors ($O(H)$).
3. `upgrade`: Check if the node is unlocked, no ancestors are locked, and the node's `locked_descendant_count` is exclusively held by the same `user_id`. If valid, unlock all descendants and lock the current node ($O(\text{Descendants})$).

Time Complexity: lock: $O(H)$, unlock: $O(H)$, upgrade: $O(N)$ in worst case (size of subtree).

Space Complexity: $O(N)$ to store the tree and metadata.

Optimizations: Use a thread-safe map for node lookups and atomics for the `locked_descendant_count` to handle high-concurrency environments.

Hints: How can we avoid checking every descendant individually during a simple `lock` operation? What metadata can a parent store about its children?

Q2: Distributed Log Sequence Recovery

Difficulty: Hard

Tags: Sliding Window, Min-Heap, Stream Processing

Problem Statement: You are receiving a high-frequency stream of logs from Azure telemetry. Each log has a unique `sequence_id` (integer) and a `timestamp`. Due to network jitter, logs arrive out of order. You are given a fixed buffer of size `K`. Your task is to output the logs in strictly increasing order

of ``sequence_id``.

Rules:

1. If the next expected ``sequence_id`` arrives, print it immediately and check the buffer for the subsequent ID.
2. If a log arrives out of order, store it in the buffer.
3. If the buffer reaches capacity ``K`` and the next expected ID is still missing, you must "skip" the missing ID, print the smallest ``sequence_id`` currently in the buffer, and update the "next expected" ID to ``smallest + 1``.

Expected Approach: Use a sorting algorithm or a simple list to store incoming logs and sort them every time the buffer fills.

Optimized Approach: Use a Min-Heap to store out-of-order logs and a Hash Set for $O(1)$ existence checks. Maintain a variable ``next_expected_id``.

1. If ``incoming_id == next_expected_id``: Print, increment ``next_expected_id``, and repeatedly pull from Min-Heap if the top matches the new ``next_expected_id``.
2. If ``incoming_id > next_expected_id``: Push to Min-Heap.
3. If ``heap.size() > K``: The next expected log is considered lost. Pop from Min-Heap, print it, set ``next_expected_id = popped_id + 1``, and check for consecutive sequences in the heap.

Time Complexity: $O(N \log K)$ where N is the number of logs.

Space Complexity: $O(K)$ for the buffer.

Optimizations: Use a pre-allocated array-based Min-Heap to reduce garbage collection overhead in high-throughput systems.

Hints: Think about how a TCP sliding window handles packet reordering. What happens to ``next_expected_id`` when we are forced to drop the gap?

Q3: Versioned Key-Value Store with Range Maximum

Difficulty: Hard

Tags: Segment Tree, Binary Search, Design, Data Structures

Problem Statement: Design a high-performance metadata store that supports three operations:

1. ``set(key, value, timestamp)``: Stores the integer ``value`` for a specific ``key`` at the given ``timestamp``.
2. ``get(key, timestamp)``: Returns the value associated with the ``key`` at the exact ``timestamp``. If the timestamp doesn't exist, return the value from the most recent timestamp prior to the input.
3. ``range_max(key, t_start, t_end)``: Returns the maximum value stored for that specific ``key`` between ``t_start`` and ``t_end`` (inclusive).

Timestamps for a specific key are guaranteed to be provided in strictly increasing order.

Expected Approach: Use a Hash Map where the key is the ``key`` string and the value is a list of pairs ``(timestamp, value)``. For ``get``, use binary search (upper bound) to find the relevant timestamp in $O(\log N)$. For ``range_max``, iterate through the list and find the maximum within the time range in $O(N)$.

Optimized Approach: Maintain a Hash Map where each key maps to a Segment Tree or a Fenwick Tree. Since timestamps are increasing, the Segment Tree can be built dynamically or represented as an array-based tree. For ``get``, continue using binary search on the list of timestamps. For ``range_max``, use the Segment Tree to query the maximum value in the index range corresponding to ``[t_start, t_end]`` (found via binary search) in $O(\log N)$.

Time Complexity: ``set``: $O(\log N)$, ``get``: $O(\log N)$, ``range_max``: $O(\log N)$, where N is the number of versions for a key.

Space Complexity: $O(N)$ to store all versions and tree nodes.

Optimizations: Use a Sparse Table for ``range_max`` if queries are frequent and the data for a key becomes immutable, or use a Square Root Decomposition for a balance between ``set`` and ``range_max`` latency.

Hints: How can we transform a time-range query into an index-range query? If timestamps are strictly increasing, does that simplify our tree construction?

Q4: Optimal Path with Single-Node Turbo Boost

Difficulty: Hard

Tags: Graph, Dynamic Programming, Directed Acyclic Graph (DAG)

Problem Statement: You are given a Directed Acyclic Graph (DAG) representing an Azure DevOps pipeline. Each node u has a weight $W[u]$ representing its execution time in minutes. The total execution time of the pipeline is the length of the longest path from any source node to any sink node. Microsoft offers a "Turbo Boost" that can be applied to exactly one node in the graph, reducing its execution time by half ($W[u] // 2$). Find the minimum possible total pipeline execution time after applying the boost to the most optimal node.

Expected Approach: For every node i in the graph, temporarily update its weight to $W[i] // 2$, calculate the longest path of the entire DAG using topological sort and DP in $O(V+E)$, and then revert the weight. Repeat for all V nodes. Total time: $O(V \cdot (V+E))$.

Optimized Approach: Perform two DP passes.

1. Compute $DistIn[u]$: The longest path ending at node u using a standard topological sort DP.
2. Compute $DistOut[u]$: The longest path starting at node u (by reversing the graph and running another topological sort DP).

The longest path passing through node u is $DistIn[u] + DistOut[u] - W[u]$.

To solve the "boost" problem, construct a 2-layered graph. Layer A contains the original nodes. Layer B contains the original nodes. For every edge $u \rightarrow v$ in the original graph, add:

- An edge $(u_A \rightarrow v_A)$ with weight $W[v]$.
- An edge $(u_B \rightarrow v_B)$ with weight $W[v]$.
- An edge $(u_A \rightarrow v_B)$ with weight $W[v] // 2$ (representing the boost).

The problem reduces to finding the longest path in this expanded DAG.

Time Complexity: $O(V + E)$ for topological sort and DP passes.

Space Complexity: $O(V + E)$ to store the graph and DP arrays.

Optimizations: Instead of an explicit 2-layered graph, use DP with state: ``dp[u][boost_used]`` where ``boost_used`` is a boolean.

Hints: If you boost a node, how does it affect paths that don't pass through it? Think about the problem as a state-space search where "boost" is a one-time transition.

Q5: Versioned Key-Value Store with Snapshot Range Queries

Difficulty: Hard

Tags: Hash Map, Binary Search, Design, Persistent Data Structures

Problem Statement: Design a system that supports `set(key, value)`, `get(key, snapshot_id)`, and `get_range_sum(start_key, end_key, snapshot_id)`. The keys are integers, and values are integers. Each `set` operation increments a global `snapshot_id`. The `get_range_sum` must return the sum of all values for keys currently existing between `start_key` and `end_key` at the time that specific `snapshot_id` was created.

Expected Approach: Maintaining a full copy of the data per snapshot is $O(N)$ space per update. A better approach uses a Map where each key points to a list of `(snapshot_id, value)` pairs.

Optimized Approach: Use a `Map<Integer, List<Pair<Integer, Integer>>>` to store versioned values for each key. To handle range sums efficiently across versions, implement a Persistent Segment Tree. Each update creates a new path in the segment tree ($O(\log N)$ nodes). This allows range queries for any version in $O(\log N)$ time.

Time Complexity: `set`: $O(\log N)$, `get`: $O(\log K)$ where K is versions per key, `get_range_sum`: $O(\log N)$.

Space Complexity: $O(U \log N)$ where U is the number of updates.

Optimizations: Use a Treap or a Skip List instead of a Segment Tree if the key range is sparse and non-contiguous.

Hints: Think about how Git stores file versions. How can we apply that to a Segment Tree?

Q6: Optimal Microservice Dependency Optimization

Difficulty: Medium/Hard

Tags: Graph, Dynamic Programming, Directed Acyclic Graph (DAG)

Problem Statement: You are given a DAG representing N microservices and their communication latencies (edges). The total latency of a request is the longest path from the entry service to any leaf service. You have a budget to optimize exactly K services. Optimizing a service reduces all its outgoing edge latencies to zero. Find the minimum possible "total latency" (longest path) after optimizing exactly K services.

Expected Approach: Use BFS/Topological sort to find the longest path, but a greedy approach for optimization fails because optimizing one node affects multiple paths.

Optimized Approach: This is a DP on DAG problem. Let $dp[u][k]$ be the minimum longest path from node u to any leaf using k optimizations. For a node u , $dp[u][k] = \min(\text{if optimized: } \max_{v \in \text{adj}(u)} (dp[v][k-1]), \text{if not optimized: } \max_{v \in \text{adj}(u)} (\text{weight}(u, v) + dp[v][k]))$.
Base case: Leaf nodes $dp[\text{leaf}][k] = 0$.

Time Complexity: $O((N + E) \cdot K)$ where N is nodes, E is edges, and K is the budget.

Space Complexity: $O(N \cdot K)$ to store the DP table.

Optimizations: Use memoization with recursion to avoid calculating unreachable states in the DP table.

Hints: If you optimize a node, its contribution to the path length becomes zero. How does this affect the recurrence relation?

Q7: Maximum Contiguous Free Memory Blocks

Difficulty: Hard

Tags: Sliding Window, Two Pointers, Greedy

Problem Statement: You are given a binary array `mem` where `0` represents a free memory block and `1` represents an allocated block. You are also given an integer `K`. You can perform at most `K` swaps between any `0` and any `1` in the array (the `0` and `1` do not need to be adjacent). Return the maximum number of contiguous `0`s you can achieve.

Expected Approach: Use a sliding window to identify a range. For any window of size `L` containing `x` ones, those `x` ones can be replaced by `0`s if two conditions are met: 1) `x <= K` (we have enough swaps) and 2) There are at least `x` zeros available outside the current window to swap in.

Optimized Approach: Calculate the total number of zeros `total\_zeros` in the array. Use a sliding window `[left, right]`. Count the number of ones `current\_ones` in the window. If `current\_ones <= K` AND `current\_ones <= (total\_zeros - (window\_size - current\_ones))`, the window is valid. Expand `right` and contract `left` to find the maximum `window\_size`.

Time Complexity: $O(N)$ where N is the length of the array.

Space Complexity: $O(1)$

Optimizations: Use a single pass to count total zeros, then a single pass for the sliding window.

Hints: The bottleneck is not just the number of swaps `K`, but the actual physical availability of zeros elsewhere in the memory to fill the gaps.

Q8: Distributed Lock Hierarchy Conflict Detector

Difficulty: Medium/Hard

Tags: Tree, Hash Map, Depth First Search

Problem Statement: In a cloud resource hierarchy (Tree), a lock on a node implicitly locks all its descendants. A node cannot be locked if any of its ancestors or any of its descendants are already locked by a different user. Design a system with `lock(node\_id, user\_id)` and `unlock(node\_id)` that manages these constraints in a large-scale tree.

Expected Approach: Maintain a hash map of `locked\_nodes`. For each `lock` request, traverse up the tree to the root to check if any ancestor is locked.

Optimized Approach: Each node in the tree structure should store: 1) `is\_locked` (boolean) and 2) `locked\_descendants\_count` (integer). To `lock(node)`: check if any ancestor is locked by traversing up; check if `locked\_descendants\_count > 0`. If both are clear, lock the node and traverse up to the root, incrementing the `locked\_descendants\_count` of all ancestors. To `unlock(node)`: decrement the `locked\_descendants\_count` for all ancestors.

Time Complexity: $O(H)$ for both lock and unlock, where H is the height of the tree.

Space Complexity: $O(N)$ to store the tree structure and lock states.

Optimizations: Use a ThreadLocal or Mutex per node for thread safety if implementing in a concurrent environment to prevent race conditions during the upward traversal.

Hints: Think about how a node can quickly know if something "below" it is locked without traversing the entire subtree. Only the path to the root needs to be updated.

Q9: Cloud Resource Allocation Optimizer

Difficulty: Hard

Tags: Segment Tree, Lazy Propagation, Greedy

Problem Statement: You are managing an Azure availability zone with N units of compute power. Customers submit reservation requests in the form $(start_time, end_time, units)$. A request can only be accepted if, for every time unit within $[start_time, end_time]$, the total units allocated (including the new request) do not exceed N . Given a sequence of M requests, return a boolean array representing whether each request was accepted or rejected.

Expected Approach: Use a segment tree where each node stores the maximum utilized capacity in a given time range. For each request, query the maximum value in $[start, end]$. If $max_val + units \leq N$, update the range by adding $units$.

Optimized Approach: Implement the segment tree with lazy propagation to handle range updates and range maximum queries in logarithmic time. Since the time range can be large (e.g., 1 to 10^9), use dynamic segment tree allocation or coordinate compression on the start/end timestamps.

Time Complexity: $O(M \log T)$ where T is the time horizon or $O(M \log M)$ with coordinate compression.

Space Complexity: $O(M)$ to store the tree nodes.

Optimizations: Use a Fenwick tree if only prefix sums were needed, but since range maximum is required, a Segment Tree is necessary. Use a Discretization technique if the number of requests is significantly smaller than the time range.

Hints: Think about how you would check if a single time point is over-capacity. How do you extend that to a range efficiently?

Q10: Distributed Service Dependency Bottleneck

Difficulty: Medium-Hard

Tags: Graph, Tarjan's Algorithm, Connectivity

Problem Statement: In a microservice architecture, services are nodes and dependencies are directed edges. A "Critical Failure Point" is defined as a service that, if it fails (along with its outgoing edges), would increase the number of disconnected components in the service mesh, potentially isolating sub-systems. Given a directed graph of V services and E dependencies, identify all "Critical Failure Points."

Expected Approach: Model the service mesh as an undirected graph (assuming bi-directional communication for heartbeats) and find all Articulation Points.

Optimized Approach: Use Tarjan's or Hopcroft-Tarjan algorithm. Maintain `discovery\_time` and `low\_link` values for each node using DFS. A node u is an articulation point if: 1. u is root and has >1 children. 2. u is not root and has a child v such that $low[v] \geq discovery[u]$.

Time Complexity: $O(V + E)$

Space Complexity: $O(V)$ for the recursion stack and metadata arrays.

Optimizations: Use an adjacency list for efficient traversal. Handle disconnected graphs by running the DFS from all unvisited nodes.

Hints: If you remove a node, how does it affect the reachability of other nodes? Look into the properties of DFS trees.
